

# SQL Mini Project – Retail Store

## TABLE STRUCTURES

### Customers

| Column Name | Data Type    | Constraints                 | Description                 |
|-------------|--------------|-----------------------------|-----------------------------|
| customer_id | INT          | PRIMARY KEY, AUTO_INCREMENT | Unique ID for each customer |
| name        | VARCHAR(100) | NOT NULL                    | Full name of the customer   |
| email       | VARCHAR(100) | UNIQUE, NOT NULL            | Unique email address        |
| phone       | VARCHAR(15)  |                             | Optional phone number       |
| created_at  | DATETIME     | DEFAULT CURRENT_TIMESTAMP   | Account creation timestamp  |

### Products

| Column Name    | Data Type     | Constraints                 | Description                          |
|----------------|---------------|-----------------------------|--------------------------------------|
| product_id     | INT           | PRIMARY KEY, AUTO_INCREMENT | Unique product ID                    |
| name           | VARCHAR(100)  | NOT NULL                    | Name of the product                  |
| category       | VARCHAR(50)   | NOT NULL                    | Product category (e.g., Electronics) |
| price          | DECIMAL(10,2) | NOT NULL CHECK (price > 0)  | Product price                        |
| stock_quantity | INT           | NOT NULL DEFAULT 0          | Units available in stock             |
| added_on       | DATE          | DEFAULT CURRENT_TIMESTAMP   | Date product was added               |

### Orders

| Column Name  | Data Type     | Constraints                                   | Description                          |
|--------------|---------------|-----------------------------------------------|--------------------------------------|
| order_id     | INT           | PRIMARY KEY, AUTO_INCREMENT                   | Unique order ID                      |
| customer_id  | INT           | FOREIGN KEY REFERENCES customers(customer_id) | Who placed the order                 |
| order_date   | DATETIME      | DEFAULT CURRENT_TIMESTAMP                     | When the order was placed            |
| status       | VARCHAR(20)   | DEFAULT 'Pending'                             | Order status (Pending, Shipped, etc) |
| total_amount | DECIMAL(10,2) |                                               | Calculated total                     |

### Order Items

| Column Name   | Data Type     | Constraints                                 | Description                        |
|---------------|---------------|---------------------------------------------|------------------------------------|
| order_item_id | INT           | PRIMARY KEY, AUTO_INCREMENT                 | Unique ID for each line item       |
| order_id      | INT           | FOREIGN KEY REFERENCES orders(order_id)     | Order to which item belongs        |
| product_id    | INT           | FOREIGN KEY REFERENCES products(product_id) | Which product was bought           |
| quantity      | INT           | NOT NULL CHECK (quantity > 0)               | Quantity ordered                   |
| item_price    | DECIMAL(10,2) | NOT NULL                                    | Snapshot of product price at order |

### Payments

| Column Name  | Data Type     | Constraints                             | Description                     |
|--------------|---------------|-----------------------------------------|---------------------------------|
| payment_id   | INT           | PRIMARY KEY, AUTO_INCREMENT             | Unique ID for payment           |
| order_id     | INT           | FOREIGN KEY REFERENCES orders(order_id) | Which order was paid for        |
| payment_date | DATETIME      | DEFAULT CURRENT_TIMESTAMP               | When payment was made           |
| amount_paid  | DECIMAL(10,2) | NOT NULL CHECK (amount_paid > 0)        | Amount paid                     |
| method       | VARCHAR(20)   | NOT NULL                                | Method (Credit Card, UPI, etc.) |

### Product Reviews

| Column Name | Data Type | Constraints                                   | Description                |
|-------------|-----------|-----------------------------------------------|----------------------------|
| review_id   | INT       | PRIMARY KEY, AUTO_INCREMENT                   | Unique ID for review       |
| product_id  | INT       | FOREIGN KEY REFERENCES products(product_id)   | Which product is reviewed  |
| customer_id | INT       | FOREIGN KEY REFERENCES customers(customer_id) | Who gave the review        |
| rating      | INT       | NOT NULL CHECK (rating BETWEEN 1 AND 5)       | Star rating                |
| review_text | TEXT      |                                               | Review message             |
| review_date | DATE      | DEFAULT CURRENT_TIMESTAMP                     | When the review was posted |

# DDL SCRIPTS

## Customers

```
CREATE TABLE customers (  
  customer_id INT PRIMARY KEY AUTO_INCREMENT,  
  name VARCHAR(100) NOT NULL,  
  email VARCHAR(100) NOT NULL UNIQUE,  
  phone VARCHAR(15),  
  created_at DATETIME DEFAULT CURRENT_TIMESTAMP  
);
```

## Products

```
CREATE TABLE products (  
  product_id INT PRIMARY KEY AUTO_INCREMENT,  
  name VARCHAR(100) NOT NULL,  
  category VARCHAR(50) NOT NULL,  
  price DECIMAL(10,2) NOT NULL,  
  stock_quantity INT NOT NULL DEFAULT 0,  
  added_on DATETIME DEFAULT CURRENT_TIMESTAMP  
);
```

## Orders

```
CREATE TABLE orders (  
  order_id INT PRIMARY KEY AUTO_INCREMENT,  
  customer_id INT,  
  order_date DATETIME DEFAULT CURRENT_TIMESTAMP,  
  status VARCHAR(20) DEFAULT 'Pending',  
  total_amount DECIMAL(10,2),  
  FOREIGN KEY (customer_id) REFERENCES customers(customer_id)  
);
```

## Order Items

```
CREATE TABLE order_items (  
  order_item_id INT PRIMARY KEY AUTO_INCREMENT,  
  order_id INT,  
  product_id INT,  
  quantity INT NOT NULL CHECK (quantity > 0),  
  item_price DECIMAL(10,2) NOT NULL,  
  FOREIGN KEY (order_id) REFERENCES orders(order_id),  
  FOREIGN KEY (product_id) REFERENCES products(product_id)  
);
```

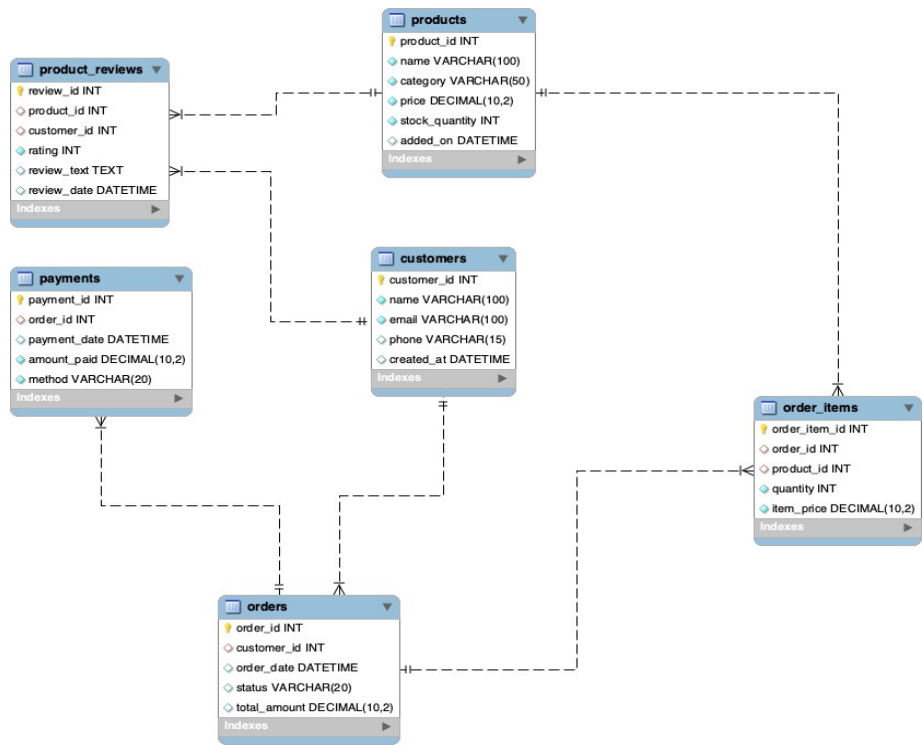
## Payments

```
CREATE TABLE payments (  
  payment_id INT PRIMARY KEY AUTO_INCREMENT,  
  order_id INT,  
  payment_date DATETIME DEFAULT CURRENT_TIMESTAMP,  
  amount_paid DECIMAL(10,2) NOT NULL CHECK (amount_paid > 0),  
  method VARCHAR(20) NOT NULL,  
  FOREIGN KEY (order_id) REFERENCES orders(order_id)  
);
```

Product Reviews

```
CREATE TABLE product_reviews (  
  review_id INT PRIMARY KEY AUTO_INCREMENT,  
  product_id INT,  
  customer_id INT,  
  rating INT NOT NULL,  
  review_text TEXT,  
  review_date DATETIME DEFAULT CURRENT_TIMESTAMP,  
  FOREIGN KEY (product_id) REFERENCES products(product_id),  
  FOREIGN KEY (customer_id) REFERENCES customers(customer_id)  
);
```

ER Diagram



Entity Relationship

| Parent Table | Child Table     | Relationship Type | Foreign Key                                         | Real-Life Meaning                             |
|--------------|-----------------|-------------------|-----------------------------------------------------|-----------------------------------------------|
| customers    | orders          | One-to-Many       | orders.customer_id → customers.customer_id          | A customer can place multiple orders          |
| orders       | order_items     | One-to-Many       | order_items.order_id → orders.order_id              | An order can contain multiple items           |
| products     | order_items     | One-to-Many       | order_items.product_id → products.product_id        | A product can appear in many different orders |
| orders       | payments        | One-to-Many       | payments.order_id → orders.order_id                 | One or more than one payment per order.       |
| products     | product_reviews | One-to-Many       | product_reviews.product_id → products.product_id    | A product can have many customer reviews      |
| customers    | product_reviews | One-to-Many       | product_reviews.customer_id → customers.customer_id | A customer can write multiple reviews         |

# Questions

## Level 1: Basics

### 1. Retrieve customer names and emails for email marketing

This helps the marketing team extract basic customer contact details for campaigns.

```
-- 1. Retrieve customer names and emails for email marketing
select name, email from customers;
```

Output-

| Result Grid        | Filter Rows:       | Export: | Wrap Cell Content: |
|--------------------|--------------------|---------|--------------------|
| name               | email              |         |                    |
| ▶ Thomas Owens     | user1@example.com  |         |                    |
| Charles Grant      | user2@example.com  |         |                    |
| Kaitlin Richards   | user3@example.com  |         |                    |
| Christina Williams | user4@example.com  |         |                    |
| David Allen        | user5@example.com  |         |                    |
| Mark Duke          | user6@example.com  |         |                    |
| Briana Wright      | user7@example.com  |         |                    |
| John Bryan         | user8@example.com  |         |                    |
| Jason Thompson     | user9@example.com  |         |                    |
| Shawn Hill         | user10@example.com |         |                    |
| Walter Jenkins     | user11@example.com |         |                    |
| Mary Knight        | user12@example.com |         |                    |
| Leslie Wilson      | user13@example.com |         |                    |
| Deborah Arias      | user14@example.com |         |                    |
| Austin Flores      | user15@example.com |         |                    |
| Amy Landry         | user16@example.com |         |                    |
| Randy Mooney       | user17@example.com |         |                    |
| Jeffrey Bray       | user18@example.com |         |                    |
| Amanda Bright      | user19@example.com |         |                    |
| Megan Lee          | user20@example.com |         |                    |

customers 1 x

### 2. View complete product catalog with all available details

The product manager may want to review all product listings in one go.

```
-- 2. View complete product catalog with all available details
select * from products;
```

Output-

Result Grid

Filter Rows:

Edit

Export/Import:

Wrap Cell Content:

| product_id | name               | category    | price   | stock_quantity | added_on            |
|------------|--------------------|-------------|---------|----------------|---------------------|
| ▶ 1        | Plant No           | Home        | 639.43  | 152            | 2024-01-30 06:30:53 |
| 2          | Population Social  | Clothing    | 4813.68 | 84             | 2025-05-30 10:02:50 |
| 3          | Available Answer   | Electronics | 2529.51 | 101            | 2025-04-13 01:11:46 |
| 4          | Any Question       | Clothing    | 4759.28 | 179            | 2025-06-03 13:34:03 |
| 5          | Natural Network    | Toys        | 4722.66 | 75             | 2023-11-06 00:47:37 |
| 6          | If Whatever        | Electronics | 177.40  | 64             | 2024-12-19 10:37:14 |
| 7          | Response Indeed    | Clothing    | 4897.36 | 36             | 2025-03-29 02:43:08 |
| 8          | Every Amount       | Home        | 4173.60 | 156            | 2025-04-30 03:11:10 |
| 9          | Common Study       | Toys        | 985.19  | 171            | 2023-07-20 13:06:42 |
| 10         | Development System | Electronics | 4801.78 | 153            | 2025-03-12 08:22:57 |
| 11         | Build Her          | Books       | 1852.64 | 150            | 2024-09-08 01:09:15 |
| 12         | Action Ask         | Electronics | 4017.01 | 19             | 2025-02-14 03:38:06 |
| 13         | Full West          | Books       | 2112.33 | 172            | 2023-09-15 03:13:38 |
| 14         | Listen Development | Home        | 296.17  | 132            | 2024-10-12 11:49:26 |
| 15         | Place Low          | Electronics | 723.97  | 46             | 2023-07-05 14:36:07 |
| 16         | Special Fact       | Toys        | 1094.18 | 15             | 2025-03-17 10:42:40 |
| 17         | Everything Plant   | Books       | 2496.68 | 120            | 2023-10-08 20:11:55 |
| 18         | Some Them          | Toys        | 3673.86 | 110            | 2024-07-25 00:33:53 |
| 19         | Build High         | Clothing    | 4707.14 | 47             | 2023-09-01 02:50:01 |
| 20         | Real Source        | Books       | 4398.66 | 197            | 2025-02-08 10:28:27 |

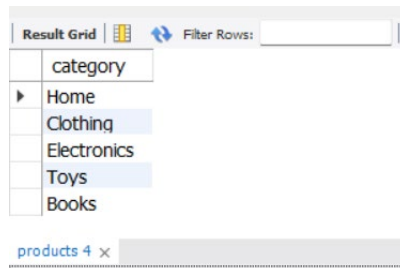
products 3 x

### 3. List all unique product categories

Useful for analyzing the range of departments or for creating filters on the website.

```
-- 3. List all unique product categories
select distinct category from products;
```

Output-



The screenshot shows a 'Result Grid' window with a 'Filter Rows' field. Below the header, a list of categories is displayed: Home, Clothing, Electronics, Toys, and Books. The 'products 4 x' tab is active at the bottom.

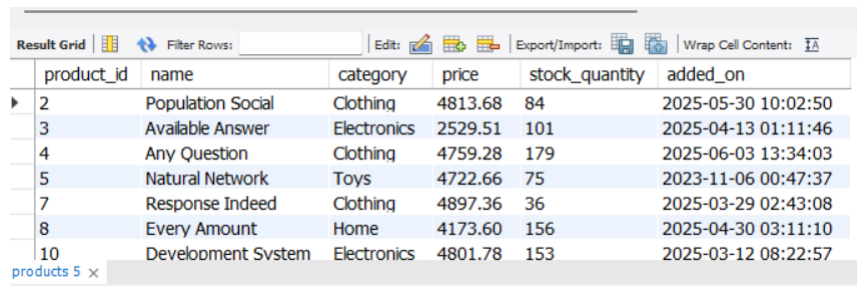
| category    |
|-------------|
| Home        |
| Clothing    |
| Electronics |
| Toys        |
| Books       |

### 4. Show all products priced above ₹1,000

This helps identify high-value items for premium promotions or pricing strategy reviews.

```
-- 4. Show all products priced above ₹1,000
select * from products where price>1000;
```

Output-



The screenshot shows a 'Result Grid' window with a 'Filter Rows' field and various tool icons. The table displays 10 products with columns: product\_id, name, category, price, stock\_quantity, and added\_on. The 'products 5 x' tab is active at the bottom.

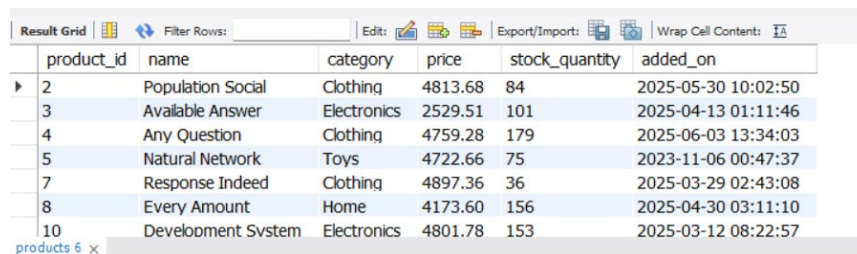
| product_id | name               | category    | price   | stock_quantity | added_on            |
|------------|--------------------|-------------|---------|----------------|---------------------|
| 2          | Population Social  | Clothing    | 4813.68 | 84             | 2025-05-30 10:02:50 |
| 3          | Available Answer   | Electronics | 2529.51 | 101            | 2025-04-13 01:11:46 |
| 4          | Any Question       | Clothing    | 4759.28 | 179            | 2025-06-03 13:34:03 |
| 5          | Natural Network    | Toys        | 4722.66 | 75             | 2023-11-06 00:47:37 |
| 7          | Response Indeed    | Clothing    | 4897.36 | 36             | 2025-03-29 02:43:08 |
| 8          | Every Amount       | Home        | 4173.60 | 156            | 2025-04-30 03:11:10 |
| 10         | Development Svstem | Electronics | 4801.78 | 153            | 2025-03-12 08:22:57 |

### 5. Display products within a mid-range price bracket (₹2,000 to ₹5,000)

A merchandising team might need this to create a mid-tier pricing campaign.

```
-- 5. Display products within a mid-range price bracket (₹2,000 to ₹5,000)
select * from products where price>=2000 and price<=5000;
select * from products where price between 2000 and 5000;
```

Output-



The screenshot shows a 'Result Grid' window with a 'Filter Rows' field and various tool icons. The table displays 10 products with columns: product\_id, name, category, price, stock\_quantity, and added\_on. The 'products 6 x' tab is active at the bottom.

| product_id | name               | category    | price   | stock_quantity | added_on            |
|------------|--------------------|-------------|---------|----------------|---------------------|
| 2          | Population Social  | Clothing    | 4813.68 | 84             | 2025-05-30 10:02:50 |
| 3          | Available Answer   | Electronics | 2529.51 | 101            | 2025-04-13 01:11:46 |
| 4          | Any Question       | Clothing    | 4759.28 | 179            | 2025-06-03 13:34:03 |
| 5          | Natural Network    | Toys        | 4722.66 | 75             | 2023-11-06 00:47:37 |
| 7          | Response Indeed    | Clothing    | 4897.36 | 36             | 2025-03-29 02:43:08 |
| 8          | Every Amount       | Home        | 4173.60 | 156            | 2025-04-30 03:11:10 |
| 10         | Development Svstem | Electronics | 4801.78 | 153            | 2025-03-12 08:22:57 |

### 6. Fetch data for specific customer IDs (e.g., from loyalty program list)

This is used when customer IDs are pre-selected from another system.

```
-- 6. Fetch data for specific customer IDs (e.g., from loyalty program list)
select * from customers where customer_id between 5 and 10;
select * from customers where customer_id in (5,6,7,8,9,10);
```



Output-

| customer_id | name           | email              | phone           | created_at          |
|-------------|----------------|--------------------|-----------------|---------------------|
| 5           | David Allen    | user5@example.com  | (751)456-8289x1 | 2023-10-29 02:43:00 |
| 6           | Mark Duke      | user6@example.com  | (144)957-2811   | 2024-06-24 03:22:59 |
| 7           | Briana Wright  | user7@example.com  | 223-833-9635    | 2023-06-25 00:35:43 |
| 8           | John Bryan     | user8@example.com  | 045.568.0798x27 | 2025-02-15 17:57:04 |
| 9           | Jason Thompson | user9@example.com  | 1862659420      | 2024-08-31 08:18:51 |
| 10          | Shawn Hill     | user10@example.com | (268)113-3152x7 | 2023-12-14 20:46:43 |

## 7. Identify customers whose names start with the letter 'A'

Used for alphabetical segmentation in outreach or app display.

```
-- 7. Identify customers whose names start with the letter 'A'  
select * from customers where name like "A%";
```

Output-

| customer_id | name           | email              | phone           | created_at          |
|-------------|----------------|--------------------|-----------------|---------------------|
| 15          | Austin Flores  | user15@example.com | 329.901.1576x66 | 2024-06-13 09:03:42 |
| 16          | Amy Landry     | user16@example.com | +1-278-019-3748 | 2024-02-28 17:51:50 |
| 19          | Amanda Bright  | user19@example.com | 380.981.9798x69 | 2024-12-20 22:58:15 |
| 27          | Adrienne Green | user27@example.com | 530.644.8455x93 | 2023-08-22 01:55:29 |

## 8. List electronics products priced under ₹3,000

Used by merchandising or frontend teams to showcase budget electronics.

```
-- 8. List electronics products priced under ₹3,000  
select * from products where category = "Electronics" and price<=3000;
```

Output-

| product_id | name             | category    | price   | stock_quantity | added_on            |
|------------|------------------|-------------|---------|----------------|---------------------|
| 3          | Available Answer | Electronics | 2529.51 | 101            | 2025-04-13 01:11:46 |
| 6          | If Whatever      | Electronics | 177.40  | 64             | 2024-12-19 10:37:14 |
| 15         | Place Low        | Electronics | 723.97  | 46             | 2023-07-05 14:36:07 |
| 31         | Series Page      | Electronics | 2070.37 | 83             | 2024-04-01 00:24:06 |
| 34         | Despite Win      | Electronics | 1340.34 | 64             | 2024-11-27 06:55:45 |
| 44         | Actually Term    | Electronics | 396.11  | 85             | 2023-11-02 13:09:20 |
| 47         | Southern Thing   | Electronics | 512.46  | 40             | 2024-02-28 17:57:38 |

## 9. Display product names and prices in descending order of price

This helps teams easily view and compare top-priced items.

```
-- 9. Display product names and prices in descending order of price  
select name, price from products order by price desc;
```

Output-

| name               | price   |
|--------------------|---------|
| Response Indeed    | 4897.36 |
| Population Social  | 4813.68 |
| Development System | 4801.78 |
| Any Question       | 4759.28 |
| Fire Often         | 4734.89 |
| Natural Network    | 4722.66 |
| Build High         | 4707.14 |
| Serious Recognize  | 4523.10 |
| Study Total        | 4413.68 |
| Real Source        | 4398.66 |
| Involve Officer    | 4244.09 |
| Group But          | 4180.23 |

## 10. Display product names and prices, sorted by price and then by name

The merchandising or catalog team may want to list products from most expensive to cheapest. If multiple products have the same price, they should be sorted alphabetically for clarity on storefronts or printed catalogs.

```
-- 10. Display product names and prices, sorted by price and then by name
select name, price from products order by price desc, name;
```

Output-

| Result Grid |                    |         | Filter Rows: | Exp |
|-------------|--------------------|---------|--------------|-----|
|             | name               | price   |              |     |
| ▶           | Response Indeed    | 4897.36 |              |     |
|             | Population Social  | 4813.68 |              |     |
|             | Development System | 4801.78 |              |     |
|             | Any Question       | 4759.28 |              |     |
|             | Fire Often         | 4734.89 |              |     |
|             | Natural Network    | 4722.66 |              |     |
|             | Build High         | 4707.14 |              |     |
|             | Serious Recognize  | 4523.10 |              |     |
|             | Study Total        | 4413.68 |              |     |
|             | Real Source        | 4398.66 |              |     |
|             | Involve Officer    | 4244.09 |              |     |
|             | Group But          | 4180.23 |              |     |

products 11 x

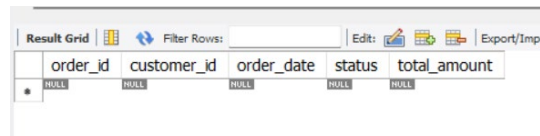
## Level 2: Filtering and Formatting

### 1. Retrieve orders where customer information is missing (possibly due to data migration or deletion)

Used to identify orphaned orders or test data where customer\_id is not linked.

```
-- 1. Retrieve orders where customer information is missing (possibly due to data migration or deletion)
select * from orders where customer_id = "Null";
```

Output-



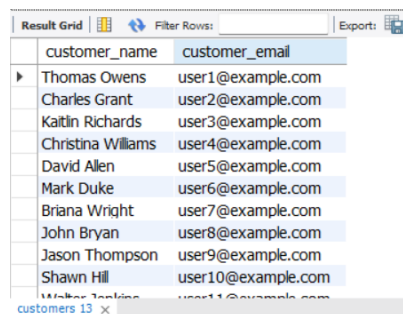
|   | order_id | customer_id | order_date | status | total_amount |
|---|----------|-------------|------------|--------|--------------|
| * | Null     | Null        | Null       | Null   | Null         |

### 2. Display customer names and emails using column aliases for frontend readability

Useful for feeding into frontend displays or report headings that require user-friendly labels.

```
-- 2. Display customer names and emails using column aliases for frontend readability
select name as customer_name, email as customer_email from customers;
```

Output-



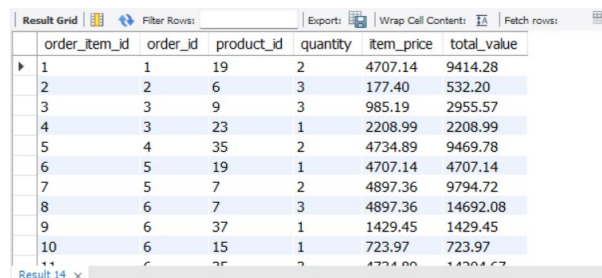
|   | customer_name      | customer_email     |
|---|--------------------|--------------------|
| ▶ | Thomas Owens       | user1@example.com  |
|   | Charles Grant      | user2@example.com  |
|   | Kaitlin Richards   | user3@example.com  |
|   | Christina Williams | user4@example.com  |
|   | David Allen        | user5@example.com  |
|   | Mark Duke          | user6@example.com  |
|   | Briana Wright      | user7@example.com  |
|   | John Bryan         | user8@example.com  |
|   | Jason Thompson     | user9@example.com  |
|   | Shawn Hill         | user10@example.com |
|   | Walter Jenkins     | user11@example.com |

### 3. Calculate total value per item ordered by multiplying quantity and item price

This can help generate per-line item bill details or invoice breakdowns.

```
-- 3. Calculate total value per item ordered by multiplying quantity and item price
select *, quantity * item_price as total_value from order_items;
```

Output-



|   | order_item_id | order_id | product_id | quantity | item_price | total_value |
|---|---------------|----------|------------|----------|------------|-------------|
| ▶ | 1             | 1        | 19         | 2        | 4707.14    | 9414.28     |
|   | 2             | 2        | 6          | 3        | 177.40     | 532.20      |
|   | 3             | 3        | 9          | 3        | 985.19     | 2955.57     |
|   | 4             | 3        | 23         | 1        | 2208.99    | 2208.99     |
|   | 5             | 4        | 35         | 2        | 4734.89    | 9469.78     |
|   | 6             | 5        | 19         | 1        | 4707.14    | 4707.14     |
|   | 7             | 5        | 7          | 2        | 4897.36    | 9794.72     |
|   | 8             | 6        | 7          | 3        | 4897.36    | 14692.08    |
|   | 9             | 6        | 37         | 1        | 1429.45    | 1429.45     |
|   | 10            | 6        | 15         | 1        | 723.97     | 723.97      |

### 4. Combine customer name and phone number in a single column

Used to show brief customer summaries or contact lists.

```
-- 4. Combine customer name and phone number in a single column
select *, concat(name,"-",phone) as customer_directory from customers;
```

Output-



| Result Grid                |             |                    |                   |                 |                     |                                    |
|----------------------------|-------------|--------------------|-------------------|-----------------|---------------------|------------------------------------|
| Filter Rows:               |             |                    |                   |                 |                     |                                    |
| Export: Wrap Cell Content: |             |                    |                   |                 |                     |                                    |
|                            | customer_id | name               | email             | phone           | created_at          | customer_directory                 |
| 1                          | 1           | Thomas Owens       | user1@example.com | 142-479-1945    | 2024-10-14 16:01:12 | Thomas Owens-142-479-1945          |
| 2                          | 2           | Charles Grant      | user2@example.com | 9153947511      | 2023-11-25 15:45:24 | Charles Grant-9153947511           |
| 3                          | 3           | Kaitlin Richards   | user3@example.com | 2073473421      | 2024-06-23 09:55:22 | Kaitlin Richards-2073473421        |
| 4                          | 4           | Christina Williams | user4@example.com | 586-605-5061x06 | 2024-10-27 17:19:38 | Christina Williams-586-605-5061x06 |
| 5                          | 5           | David Allen        | user5@example.com | (751)456-8289x1 | 2023-10-29 02:43:00 | David Allen-(751)456-8289x1        |
| 6                          | 6           | Mark Duke          | user6@example.com | (144)957-2811   | 2024-06-24 03:22:59 | Mark Duke-(144)957-2811            |
| 7                          | 7           | Briana Wright      | user7@example.com | 223-833-9635    | 2023-06-25 00:35:43 | Briana Wright-223-833-9635         |
| 8                          | 8           | John Bryan         | user8@example.com | 045.568.0798x27 | 2025-02-15 17:57:04 | John Bryan-045.568.0798x27         |

## 5. Extract only the date part from order timestamps for date-wise reporting

Helps group or filter orders by date without considering time.

```
-- 5. Extract only the date part from order timestamps for date-wise reporting
```

```
select *, date(order_date) as date from orders order by order_date;
```

Output-

| Result Grid                |          |             |                     |           |              |            |
|----------------------------|----------|-------------|---------------------|-----------|--------------|------------|
| Filter Rows:               |          |             |                     |           |              |            |
| Export: Wrap Cell Content: |          |             |                     |           |              |            |
|                            | order_id | customer_id | order_date          | status    | total_amount | date       |
| 18                         | 18       | 13          | 2024-06-15 12:57:04 | Cancelled | 32001.24     | 2024-06-15 |
| 199                        | 199      | 30          | 2024-06-18 19:01:34 | Delivered | 4993.36      | 2024-06-18 |
| 24                         | 24       | 6           | 2024-06-19 07:09:04 | Pending   | 22882.19     | 2024-06-19 |
| 66                         | 66       | 27          | 2024-06-19 09:41:16 | Pending   | 26176.05     | 2024-06-19 |
| 84                         | 84       | 28          | 2024-06-19 23:41:28 | Cancelled | 19007.10     | 2024-06-19 |
| 208                        | 208      | 7           | 2024-06-20 02:14:47 | Pending   | 9469.78      | 2024-06-20 |
| 160                        | 160      | 18          | 2024-06-20 16:15:08 | Delivered | 28077.66     | 2024-06-20 |
| 175                        | 175      | 30          | 2024-06-21 00:47:44 | Shipped   | 15372.41     | 2024-06-21 |

## 6. List products that do not have any stock left

This helps the inventory team identify out-of-stock items.

```
-- 6. List products that do not have any stock left
```

```
select * from products where stock_quantity = 0;
```

Output-

| Result Grid               |            |      |          |       |                |          |
|---------------------------|------------|------|----------|-------|----------------|----------|
| Filter Rows:              |            |      |          |       |                |          |
| Edit: Export/Import: Wrap |            |      |          |       |                |          |
|                           | product_id | name | category | price | stock_quantity | added_on |
| *                         | NULL       | NULL | NULL     | NULL  | NULL           | NULL     |

## Level 3: Aggregations

### 1. Count the total number of orders placed

Used by business managers to track order volume over time.

```
-- 1. Count the total number of orders placed
select count(order_id) as total_orders from orders;
```

Output-

| Result Grid |              | Filter Rows: |
|-------------|--------------|--------------|
|             | total_orders |              |
| ▶           | 400          |              |

### 2. Calculate the total revenue collected from all orders

This gives the overall sales value.

```
-- 2. Calculate the total revenue collected from all orders
select sum(total_amount) as total_revenue from orders;
```

Output-

| Result Grid |               | Filter Rows: |
|-------------|---------------|--------------|
|             | total_revenue |              |
| ▶           | 6960973.66    |              |

### 3. Calculate the average order value

Used for understanding customer spending patterns.

```
-- 3. Calculate the average order value
select avg(total_amount) as Avg_order from orders;
```

Output-

| Result Grid |              | Filter Rows: |
|-------------|--------------|--------------|
|             | Avg_order    |              |
| ▶           | 17402.434150 |              |

### 4. Count the number of customers who have placed at least one order

This identifies active customers.

```
-- 4. Count the number of customers who have placed at least one order
select count(distinct customer_id) as no_of_customers from orders;
```

Output-

| Result Grid |                 | Filter Rows: |
|-------------|-----------------|--------------|
|             | no_of_customers |              |
| ▶           | 30              |              |

### 5. Find the number of orders placed by each customer

Helpful for identifying top or repeat customers.

```
-- 5. Find the number of orders placed by each customer
select customer_id, count(customer_id) as no_of_orders from orders group by customer_id;
```

Output-



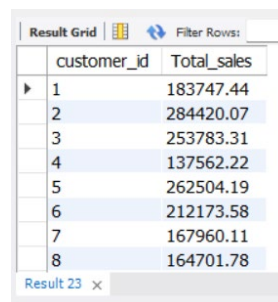
| customer_id | no_of_orders |
|-------------|--------------|
| 1           | 12           |
| 2           | 17           |
| 3           | 17           |
| 4           | 9            |
| 5           | 14           |
| 6           | 16           |
| 7           | 13           |
| 8           | 10           |

## 6. Find total sales amount made by each customer

-- 6. Find total sales amount made by each customer

```
select customer_id, sum(total_amount) as Total_sales from orders group by customer_id;
```

Output-



| customer_id | Total_sales |
|-------------|-------------|
| 1           | 183747.44   |
| 2           | 284420.07   |
| 3           | 253783.31   |
| 4           | 137562.22   |
| 5           | 262504.19   |
| 6           | 212173.58   |
| 7           | 167960.11   |
| 8           | 164701.78   |

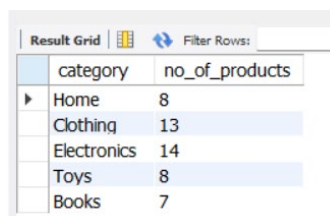
## 7. List the number of products sold per category

This helps category managers assess performance by department.

-- 7. List the number of products sold per category

```
select category, count(*) as no_of_products from products group by category;
```

Output-



| category    | no_of_products |
|-------------|----------------|
| Home        | 8              |
| Clothing    | 13             |
| Electronics | 14             |
| Toys        | 8              |
| Books       | 7              |

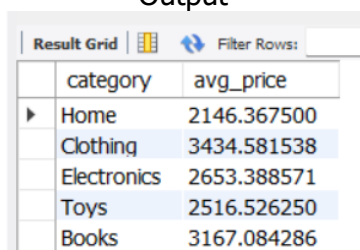
## 8. Find the average item price per category

Useful to compare pricing across departments.

-- 8. Find the average item price per category

```
select category, avg(price) as avg_price from products group by category;
```

Output-



| category    | avg_price   |
|-------------|-------------|
| Home        | 2146.367500 |
| Clothing    | 3434.581538 |
| Electronics | 2653.388571 |
| Toys        | 2516.526250 |
| Books       | 3167.084286 |

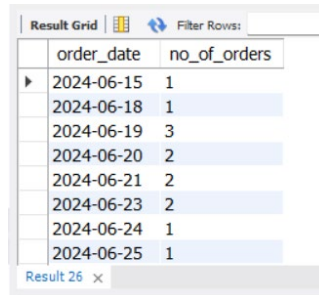
## 9. Show number of orders placed per day

Used to track daily business activity and demand trends.

```
-- 9. Show number of orders placed per day
```

```
select date(order_date) as order_date, count(*) as no_of_orders from orders
group by date(order_date) order by date(order_date);
```

Output-



The screenshot shows a 'Result Grid' window with a 'Filter Rows' search bar. The table has two columns: 'order\_date' and 'no\_of\_orders'. The data is as follows:

| order_date | no_of_orders |
|------------|--------------|
| 2024-06-15 | 1            |
| 2024-06-18 | 1            |
| 2024-06-19 | 3            |
| 2024-06-20 | 2            |
| 2024-06-21 | 2            |
| 2024-06-23 | 2            |
| 2024-06-24 | 1            |
| 2024-06-25 | 1            |

At the bottom, it says 'Result 26 x'.

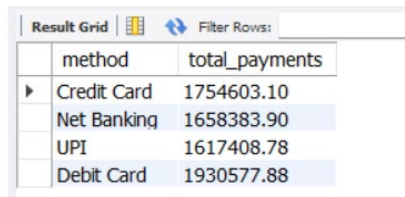
## 10. List total payments received per payment method

Helps the finance team understand preferred transaction modes.

```
-- 10. List total payments received per payment method
```

```
select method, sum(amount_paid) as total_payments from payments group by method;
```

Output-



The screenshot shows a 'Result Grid' window with a 'Filter Rows' search bar. The table has two columns: 'method' and 'total\_payments'. The data is as follows:

| method      | total_payments |
|-------------|----------------|
| Credit Card | 1754603.10     |
| Net Banking | 1658383.90     |
| UPI         | 1617408.78     |
| Debit Card  | 1930577.88     |

## Level 4: Multi-Table Queries (JOINS)

### 1. Retrieve order details along with the customer name (INNER JOIN)

Used for displaying which customer placed each order.

```
-- 1. Retrieve order details along with the customer name (INNER JOIN)
select c.name, o.*
from customers c
inner join orders o
on c.customer_id = o.customer_id;
```

Output-

|   | name         | order_id | customer_id | order_date          | status  | total_amount |
|---|--------------|----------|-------------|---------------------|---------|--------------|
| ▶ | Thomas Owens | 14       | 1           | 2024-09-24 21:21:38 | Shipped | 15803.34     |
|   | Thomas Owens | 17       | 1           | 2024-08-19 21:17:57 | Pending | 11173.77     |
|   | Thomas Owens | 61       | 1           | 2024-12-26 23:21:58 | Shipped | 13053.00     |
|   | Thomas Owens | 76       | 1           | 2025-01-14 22:59:54 | Pending | 23506.81     |
|   | Thomas Owens | 92       | 1           | 2024-09-26 11:33:58 | Shipped | 834.75       |
|   | Thomas Owens | 109      | 1           | 2025-03-17 04:56:18 | Shipped | 12190.14     |
|   | Thomas Owens | 127      | 1           | 2025-06-10 18:01:32 | Pending | 20160.64     |
|   | Thomas Owens | 135      | 1           | 2025-03-11 22:58:23 | Pending | 8891.79      |

### 2. Get list of products that have been sold (INNER JOIN with order\_items)

Used to find which products were actually included in orders.

```
-- 2. Get list of products that have been sold (INNER JOIN with order_items)
select distinct p.name
from products p
inner join order_items i
on p.product_id = i.product_id;
```

Output-

|   | name              |
|---|-------------------|
| ▶ | Plant No          |
|   | Population Social |
|   | Available Answer  |
|   | Any Question      |
|   | Natural Network   |
|   | If Whatever       |
|   | Response Indeed   |
|   | Every Amount      |

### 3. List all orders with their payment method (INNER JOIN)

Used by finance or audit teams to see how each order was paid for.

```
-- 3. List all orders with their payment method (INNER JOIN)
select o.order_id, p.method
from orders o
inner join payments p
on o.order_id = p.payment_id
order by o.order_id;
```

Output-

| Result Grid |          | Filter Rows: |
|-------------|----------|--------------|
|             | order_id | method       |
| ▶           | 1        | Credit Card  |
|             | 2        | Net Banking  |
|             | 3        | Credit Card  |
|             | 4        | UPI          |
|             | 5        | UPI          |
|             | 6        | UPI          |
|             | 7        | UPI          |
|             | 8        | Net Banking  |

#### 4. Get list of customers and their orders (LEFT JOIN)

Used to find all customers and see who has or hasn't placed orders.

```
-- 4. Get list of customers and their orders (LEFT JOIN)
select c.name, o.*
from customers c
left join orders o
on c.customer_id = o.customer_id;
```

Output-

Result Grid

Filter Rows:

Export:

Wrap Cell Contents: [FA](#)

|   | name         | order_id | customer_id | order_date          | status  | total_amount |
|---|--------------|----------|-------------|---------------------|---------|--------------|
| ▶ | Thomas Owens | 14       | 1           | 2024-09-24 21:21:38 | Shipped | 15803.34     |
|   | Thomas Owens | 17       | 1           | 2024-08-19 21:17:57 | Pending | 11173.77     |
|   | Thomas Owens | 61       | 1           | 2024-12-26 23:21:58 | Shipped | 13053.00     |
|   | Thomas Owens | 76       | 1           | 2025-01-14 22:59:54 | Pending | 23506.81     |
|   | Thomas Owens | 92       | 1           | 2024-09-26 11:33:58 | Shipped | 834.75       |
|   | Thomas Owens | 109      | 1           | 2025-03-17 04:56:18 | Shipped | 12190.14     |
|   | Thomas Owens | 127      | 1           | 2025-06-10 18:01:32 | Pending | 20160.64     |
|   | Thomas Owens | 135      | 1           | 2025-03-11 22:58:23 | Pending | 8891.79      |

Result 31 ×

#### 5. List all products along with order item quantity (LEFT JOIN)

Useful for inventory teams to track what sold and what hasn't.

```
-- 5. List all products along with order item quantity (LEFT JOIN)
select p.name, count(i.quantity) as item_quantity
from products p
left join order_items i
on p.product_id = i.product_id
group by p.name;
```

Output-

| Result Grid |                   | Filter Rows:  |
|-------------|-------------------|---------------|
|             | name              | item_quantity |
| ▶           | Plant No          | 22            |
|             | Population Social | 16            |
|             | Available Answer  | 25            |
|             | Any Question      | 13            |
|             | Natural Network   | 21            |
|             | If Whatever       | 27            |
|             | Response Indeed   | 21            |
|             | Every Amount      | 27            |



## 6. List all payments including those with no matching orders (RIGHT JOIN)

Rare but used when ensuring all payments are mapped correctly.

```
-- 6. List all payments including those with no matching orders (RIGHT JOIN)
select *
from payments p
right join orders o
on p.order_id = o.order_id;
```

Output-

|   | payment_id | order_id | payment_date        | amount_paid | method      | order_id | customer_id | order_date          | status    | total_amc |
|---|------------|----------|---------------------|-------------|-------------|----------|-------------|---------------------|-----------|-----------|
| ▶ | 1          | 1        | 2025-03-02 08:16:11 | 9414.28     | Credit Card | 1        | 20          | 2025-03-02 07:20:11 | Delivered | 9414.28   |
|   | 2          | 2        | 2024-10-09 19:00:21 | 532.20      | Net Banking | 2        | 18          | 2024-10-09 18:08:21 | Shipped   | 532.20    |
|   | 3          | 3        | 2025-05-08 00:55:27 | 5164.56     | Credit Card | 3        | 15          | 2025-05-08 00:08:27 | Cancelled | 5164.56   |
|   | 4          | 4        | 2024-09-19 22:28:13 | 9469.78     | UPI         | 4        | 11          | 2024-09-19 22:16:13 | Delivered | 9469.78   |
|   | 5          | 5        | 2025-04-08 18:26:06 | 14501.86    | UPI         | 5        | 12          | 2025-04-08 18:02:06 | Pending   | 14501.86  |
|   | 6          | 6        | 2024-10-25 08:16:59 | 31050.17    | UPI         | 6        | 29          | 2024-10-25 07:33:59 | Cancelled | 31050.17  |
|   | 7          | 7        | 2024-07-29 12:45:47 | 3043.67     | UPI         | 7        | 22          | 2024-07-29 11:58:47 | Shipped   | 3043.67   |

## 7. Combine data from three tables: customer, order, and payment

Used for detailed transaction reports.

```
-- 7. Combine data from three tables: customer, order, and payment
select *
from customers c
join orders o
on c.customer_id = o.customer_id
join payments p
on o.order_id = p.order_id;
```

Output-

|   | customer_id | name         | email             | phone        | created_at          | order_id | customer_id | order_date          | stat |
|---|-------------|--------------|-------------------|--------------|---------------------|----------|-------------|---------------------|------|
| ▶ | 1           | Thomas Owens | user1@example.com | 142-479-1945 | 2024-10-14 16:01:12 | 14       | 1           | 2024-09-24 21:21:38 | Ship |
|   | 1           | Thomas Owens | user1@example.com | 142-479-1945 | 2024-10-14 16:01:12 | 17       | 1           | 2024-08-19 21:17:57 | Pen  |
|   | 1           | Thomas Owens | user1@example.com | 142-479-1945 | 2024-10-14 16:01:12 | 61       | 1           | 2024-12-26 23:21:58 | Ship |
|   | 1           | Thomas Owens | user1@example.com | 142-479-1945 | 2024-10-14 16:01:12 | 76       | 1           | 2025-01-14 22:59:54 | Pen  |
|   | 1           | Thomas Owens | user1@example.com | 142-479-1945 | 2024-10-14 16:01:12 | 92       | 1           | 2024-09-26 11:33:58 | Ship |
|   | 1           | Thomas Owens | user1@example.com | 142-479-1945 | 2024-10-14 16:01:12 | 109      | 1           | 2025-03-17 04:56:18 | Ship |
|   | 1           | Thomas Owens | user1@example.com | 142-479-1945 | 2024-10-14 16:01:12 | 127      | 1           | 2025-06-10 18:01:32 | Pen  |

## Level 5: Subqueries (Inner Queries)

### 1. List all products priced above the average product price

Used by pricing analysts to identify premium-priced products.

```
-- 1. List all products priced above the average product price
select * from products where price > (select avg(price) from products);
```

Output-

|   | product_id | name               | category    | price   | stock_quantity | added_on            |
|---|------------|--------------------|-------------|---------|----------------|---------------------|
| ▶ | 2          | Population Social  | Clothing    | 4813.68 | 84             | 2025-05-30 10:02:50 |
|   | 4          | Any Question       | Clothing    | 4759.28 | 179            | 2025-06-03 13:34:03 |
|   | 5          | Natural Network    | Toys        | 4722.66 | 75             | 2023-11-06 00:47:37 |
|   | 7          | Response Indeed    | Clothing    | 4897.36 | 36             | 2025-03-29 02:43:08 |
|   | 8          | Every Amount       | Home        | 4173.60 | 156            | 2025-04-30 03:11:10 |
|   | 10         | Development System | Electronics | 4801.78 | 153            | 2025-03-12 08:22:57 |
|   | 12         | Action Ask         | Electronics | 4017.01 | 19             | 2025-02-14 03:38:06 |
|   | 18         | Some Them          | Toys        | 3673.86 | 110            | 2024-07-25 00:33:53 |

products 35 x

### 2. Find customers who have placed at least one order

Used to identify active customers for loyalty campaigns.

```
-- 2. Find customers who have placed at least one order
select count(distinct customer_id) as no_of_customers from orders;
```

Output-

|   | no_of_customers |
|---|-----------------|
| ▶ | 30              |

### 3. Show orders whose total amount is above the average for that customer

Used to detect unusually high purchases per customer.

```
-- 3. Show orders whose total amount is above the average for that customer
select * from
(select *, avg(total_amount) over(partition by customer_id) as avg_amount from orders) t
where total_amount > avg_amount;
```

Output-

|   | order_id | customer_id | order_date          | status    | total_amount | avg_amount   |
|---|----------|-------------|---------------------|-----------|--------------|--------------|
| ▶ | 14       | 1           | 2024-09-24 21:21:38 | Shipped   | 15803.34     | 15312.286667 |
|   | 76       | 1           | 2025-01-14 22:59:54 | Pending   | 23506.81     | 15312.286667 |
|   | 127      | 1           | 2025-06-10 18:01:32 | Pending   | 20160.64     | 15312.286667 |
|   | 278      | 1           | 2025-02-01 20:05:38 | Delivered | 32015.16     | 15312.286667 |
|   | 379      | 1           | 2025-03-22 15:57:58 | Cancelled | 21586.89     | 15312.286667 |
|   | 223      | 2           | 2024-11-25 01:32:18 | Cancelled | 41020.75     | 16730.592353 |
|   | 230      | 2           | 2024-07-25 22:19:17 | Pending   | 22655.32     | 16730.592353 |
|   | 315      | 2           | 2025-01-27 15:23:27 | Cancelled | 42056.04     | 16730.592353 |

Result 37 x

### 4. Display customers who haven't placed any orders

Used for re-engagement campaigns targeting inactive users.

```
-- 4. Display customers who haven't placed any orders
select name from customers where customer_id not in (select customer_id from orders);
```

Output-

|  | name |
|--|------|
|  |      |

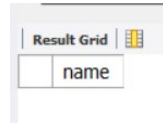
## 5. Show products that were never ordered

Helps with inventory clearance decisions or product deactivation.

```
-- 5. Show products that were never ordered
```

```
select name from products where product_id not in (select product_id from order_items);
```

Output-



| name |
|------|
|------|

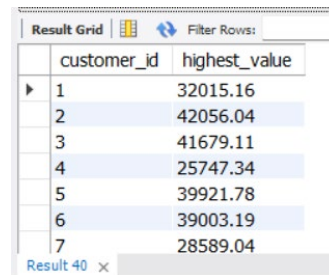
## 6. Show highest value order per customer

Used to identify the largest transaction made by each customer.

```
-- 6. Show highest value order per customer
```

```
select customer_id, max(total_amount) as highest_value from orders group by customer_id;
```

Output-



| customer_id | highest_value |
|-------------|---------------|
| 1           | 32015.16      |
| 2           | 42056.04      |
| 3           | 41679.11      |
| 4           | 25747.34      |
| 5           | 39921.78      |
| 6           | 39003.19      |
| 7           | 28589.04      |

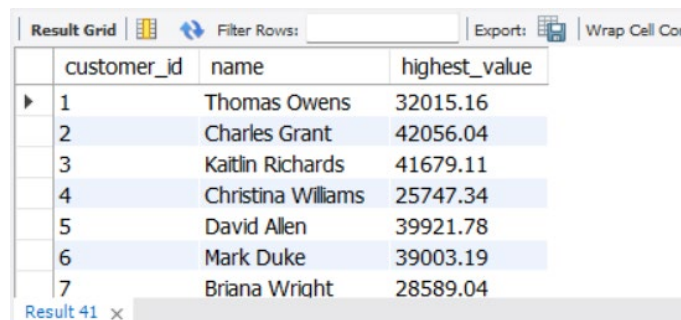
## 7. Highest Order Per Customer (Including Names)

Used to identify the largest transaction made by each customer. Outputs name as well.

```
-- 7. Highest Order Per Customer (Including Names)
```

```
select c.customer_id, c.name, max(o.total_amount) as highest_value  
from customers c  
left join orders o  
on c.customer_id = o.customer_id  
group by c.customer_id;
```

Output-



| customer_id | name               | highest_value |
|-------------|--------------------|---------------|
| 1           | Thomas Owens       | 32015.16      |
| 2           | Charles Grant      | 42056.04      |
| 3           | Kaitlin Richards   | 41679.11      |
| 4           | Christina Williams | 25747.34      |
| 5           | David Allen        | 39921.78      |
| 6           | Mark Duke          | 39003.19      |
| 7           | Briana Wright      | 28589.04      |

## Level 6: Set Operations

### 1. List all customers who have either placed an order or written a product review

Used to identify engaged customers from different activity areas.

```
-- 1. List all customers who have either placed an order or written a product review
```

```
select customer_id from orders
union
select customer_id from product_reviews;
```

Output-



The screenshot shows a 'Result Grid' window with a 'Filter' button. The grid contains a single column labeled 'customer\_id' with 7 rows of data: 1, 2, 3, 4, 5, 6, and 7. The rows are alternatingly highlighted in white and light blue. At the bottom of the grid, it says 'Result 42 x'.

| customer_id |
|-------------|
| 1           |
| 2           |
| 3           |
| 4           |
| 5           |
| 6           |
| 7           |

### 2. List all customers who have placed an order as well as reviewed a product

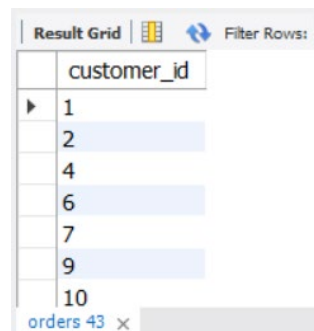
**[intersect not supported]**

Used to identify highly engaged customers for rewards.

```
-- 2. List all customers who have placed an order as well as reviewed a product [intersect not supported]
```

```
select distinct customer_id from orders
where customer_id in
(select customer_id from product_reviews);
```

Output-



The screenshot shows a 'Result Grid' window with a 'Filter Rows:' button. The grid contains a single column labeled 'customer\_id' with 6 rows of data: 1, 2, 4, 6, 7, and 9. The rows are alternatingly highlighted in white and light blue. At the bottom of the grid, it says 'orders 43 x'.

| customer_id |
|-------------|
| 1           |
| 2           |
| 4           |
| 6           |
| 7           |
| 9           |